

04 / 25

AI Skills for Real Engineers · 7 min read · Updated Aug 24, 2026



The /to-spec Skill

Turn an agreed conversation into a written spec.



Matt Pocock

Subscribe

 Copy page Share

Next page →

Install this skill

```
$ npx skills@latest add mattpocock/skills --skill=to-spec
```



Then type `/to-spec` in your coding agent.

Source

mattpocock/skills

≡ On this page



What it does

`to-spec` turns the conversation you have just had into a spec, and publishes it to your issue tracker as a single issue.

It does not interview you. By the time you reach for it the deciding is already done, so it synthesises what is known (from the thread, from the codebase, from your `CONTEXT.md` and ADRs) rather than opening a fresh round of questions. The spec is a record of decisions already made, not a place where new ones get made.

≡ Pages

When to reach for it

You invoke this by typing `/to-spec` ; the agent won't reach for it on its own.

Reach for it when the build is too big for one agent session and has to survive being split across several. That is the whole trigger:

Where you are	What to run	
You haven't decided anything yet	<code>grill-with-docs</code> first	
Decided, and the work fits one context window	<code>implement</code> : skip the spec	
Decided, and the work spans several sessions	<code>/to-spec</code> , then <code>to-tickets</code>	
A <u>wayfinder</u> map has cleared	<code>/to-spec #<map_issue></code>	

Prerequisites

`to-spec` publishes the spec as an issue, so setup-matt-pocock-skills must have configured a tracker and the triage-label vocabulary for this repo first. Either kind works: a real tracker like GitHub, or local markdown files under `.scratch/` , which is supported out of the box.

The spec is a decision record

The spec exists because context windows end. Everything you settled while grilling (the shape of the solution, the choices you argued through, what you deliberately refused) is in one conversation that is about to be cleared. The spec is what survives that.

So it does not validate anything, and it does not decide anything. It captures what was decided, in your project's own vocabulary, so that a fresh session can pick the work up without you re-explaining it. Anything the spec asserts that you never actually said is a defect.

Seams before prose

Before it writes a word, `to-spec` sketches the **seams** the feature will be tested at, and checks them with you. It prefers seams that already exist to new ones, and takes the highest seam it can: the ideal number across a change is one.

Those agreed seams then travel. tdd works only at pre-agreed seams, and code-review reviews the diff against the spec, so a seam nobody agreed to shows up as a review finding. The binding is indirect: it runs through this document, which is exactly why the seam conversation is worth taking seriously here rather than deferring it to implementation.

Common questions

Where did `/to-prd` go? It is this skill, renamed in v1.1. "Spec" is now the single through-line term, and the old `to-prd` slug is dead; reinstall under the new name. The pair that replaced the old vocabulary is *spec* and *tickets*: the spec is the destination and the decisions that fix it, the tickets are the execution steps that get there. If you pivot, delete the unfinished tickets and keep the spec.

Why does the spec get the `ready-for-agent` label? I don't want an agent implementing off it. The label means "no further triage needed": the document is complete enough for an agent to work from. It is an input designation, not a work order. But if you run AFK agents that poll for `ready-for-agent`, that distinction isn't visible to them, and they will happily try to build the whole spec in one run instead of picking up the ticket slices. This is the most-reported rough edge on the skill. Until it changes, exclude the parent spec explicitly in your AFK agent's prompt, or strip the label once `/to-tickets` has run.

Why not go straight from grilling to `/to-tickets` and skip the spec?

Often you should; the spec earns its step only on multi-session work.

Where it pays is that the tickets are disposable and the spec isn't: each ticket is sized for one fresh context window and gets deleted or closed, while the spec stays as the one place the reasoning behind them lives. On a single-session change that buys you nothing, and you have paid an extra synthesis step where the model can drift. Go grilling → /implement.

I just finished a wayfinder map. What do I feed it? The main map issue: /to-spec #<map_issue>, not the individual decision tickets. wayfinder produces decisions rather than deliverables, scattered across a map; to-spec is the step that collapses them into one buildable document. Looping the map straight into /implement throws that collapse away.

Is the spec for me to review, or is it just for the agent? Mostly for the agent, and it reads that way: complete, dense, reference-heavy. The parts worth your eyes are the seams and the out-of-scope section, because those are the two places a wrong decision is cheapest to catch and most expensive to discover later. Reading the whole thing end to end is a real complaint people have, and there is no summary mode: the honest answer is that if the spec surprises you, the grilling was too shallow, not the spec too long.

Do I keep the spec frozen once tickets start, or let the agent rewrite it? Nothing keeps it in sync, so in practice it is a snapshot of what you knew at that moment, and it goes stale the first time implementation teaches you something. Treat it as throwaway once the work ships. The artifacts meant to outlive it are your CONTEXT.md and your ADRs; if something learned during implementation deserves to last, it belongs there, not in an edited spec.

My work is a refactor or a module boundary, not a feature. Does the template fit? Less well, and this is a known limitation. The template leans hard on user stories, which is the wrong shape for architectural work: you end up writing stories nobody asked for around decisions that are really about interfaces and invariants. Lean on the implementation-decisions and testing-decisions sections instead, and let the durable architectural

calls land as ADRs via grill-with-docs rather than trying to make the spec carry them.

Will it check the tracker for related work, or cite the ADRs it's

respecting? No to both. It reads and respects the ADRs covering the area it touches, but it doesn't link them, and it doesn't search the tracker for overlapping issues before drafting, so a spec can quietly duplicate work someone already filed. Search the tracker yourself first if the area is busy.

`/to-tickets` **couldn't read my spec: it kept truncating.** Very large specs can outgrow what a tracker issue will serve back cleanly, and there is no local copy to fall back on. The fix is context hygiene: don't clear or compact between `/to-spec` and `/to-tickets`. Run them in the same window and the spec never has to be re-fetched at all.

It's working if

It starts writing rather than asking you a fresh round of questions.

It puts the seams to you before it writes, and proposes as few as it can get away with.

It comes back in your project's nouns, not generic product-management boilerplate.

Every decision in it is one you can remember making. Nothing was invented to fill a section.

The out-of-scope section has real things in it: the things you refused are usually the most useful lines on the page.

Where it fits

`to-spec` is a step in the main build chain, and only on the multi-session branch of it:

grill-with-docs → to-spec → to-tickets → implement → code-review

Its neighbours upstream are grill-with-docs, which does the deciding this skill only records, and wayfinder, whose finished map merges onto the chain right here. Downstream, to-tickets cuts the spec into tracer-bullet tickets for implement to build. When you're unsure which skill or flow fits, ask-matt routes you.

AI Hero · Free email course

A clear spec can still be too big to hand to one agent.

One command installs all 22 skills. Seven free lessons turn a clear spec into reviewable tickets, one sandboxed run, and a checked diff. Go at your pace.

Start the free course

✓ I respect your privacy. Unsubscribe at any time.

Skill actions

Install the skills

▲ Skills 21.7M

All agents

Claude Code

```
$ npx skills@latest add mattpocock/skills
```



Installs the whole set. Then type `/to-spec` in your coding agent.

Update with `npx skills update`

Skills.sh ↗

Previous skill

← /grill-with-docs

Next skill

/to-tickets



Share



Related reading

Skill · 8 min read

The /prototype Skill



Video

triage: Turn Backlog Mess Into Agent-Ready Work

